UNIVERSIDAD EAFIT Ingeniería de Sistemas

SI3003 — Inteligencia Artificial

Evaluación 2 — Parte II (aplicada)

Profesor: Jorge Iván Padilla Buriticá **Semestre:** _____

Nombre: _____ **Documento:** _____

PARTE II — APLICADA

Notebook de Python · IA permitida · tiempo máximo: 60 minutos · 100 puntos

Reglas

- Se permite **usar IA, documentación e internet** y **reutilizar el código de los notebooks vistos en clase**. Usted es responsable de que el código funcione y de **entender y explicar** cada resultado.
- La Parte II está diseñada para que **no haya que entrenar modelos grandes ni esperar descargas largas**: los resultados de YOLO, el corpus y los embeddings ya están calculados y se entregan en la carpeta `datos_examen/`. Lo que se evalúa es el código **correcto, verificado** y sobre todo la **interpretación** de sus resultados.
- La Parte I (teórica) ya fue entregada. Entregable: **un único notebook** `Eval2_Apellido_Nombre.ipynb`, **ejecutado de arriba hacia abajo** con las salidas visibles.
- **Entrega por correo, hasta las 11:00 a. m.:** envíe el notebook a `jipadillab@eafit.edu.co` con el asunto

  `[SI3003-Nombre_Apellido_Grupo_Eval02_practica]`

  Los análisis se escriben en celdas **Markdown** dentro del notebook.
- Fije `SEED` = últimos 4 dígitos de su documento (`random`, `numpy`, `torch`). Sea d su **último dígito** de documento: define el subconjunto de imágenes de la Sección A.3.
- **Credenciales:** cargue sus API keys (`NVIDIA_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`) desde *Colab Secrets* o `.env`. Ninguna celda ni salida del notebook entregado puede mostrar una key; si ocurre, se penaliza con 5 puntos.
- **Proveedores permitidos:** puede usar modelos de **NVIDIA NIM, Groq, Hugging Face** u otro proveedor compatible con el SDK `openai`, y mezclar proveedores (p. ej. chat en Groq y embeddings locales), **siempre que respete las reglas de compatibilidad del recuadro siguiente**. Los resultados que se piden no dependen del proveedor elegido.
- **Tiempo sugerido:** A (18 min), B (15 min), C (22 min), D (5 min). Si una tarea se complica, avance y deje la celda explicando qué intentó.

Proveedores y compatibilidad (léalo antes de empezar)

Los tres proveedores exponen **chat** con el mismo SDK: `from openai import OpenAI`, cambiando solo `base_url`, la clave y el nombre del modelo. **No todos ofrecen embeddings.** Confirme los modelos disponibles con `client.models.list()`: los catálogos cambian.

Proveedor	Chat	Embeddings	Base URL y notas
NVIDIA NIM	Sí	Sí	`https://integrate.api.nvidia.com/v1`. Los modelos de embeddings asimétricos (p. ej. `nvidia/nemotron-3-embed-1b`) exigen `input_type`: `passage` para documentos y `query` para consultas, enviado con `extra_body`. `enable_thinking` solo aplica a Nemotron.
Groq	Sí	**No**	`https://api.groq.com/openai/v1` (el segmento `/openai` es obligatorio). No acepta `logprobs`, `logit_bias`, `top_logprobs` ni `messages[].name`; `n` debe ser 1.
Hugging Face	Sí	No por este endpoint	`https://router.huggingface.co/v1`; clave `HF_TOKEN`; el nombre del modelo admite un sufijo de proveedor (`:fastest`, `:cheapest`). Embeddings: `huggingface_hub.InferenceClient.feature_extraction` o `sentence-transformers` local.
Local	Sí (lento)	Sí	`transformers` (p. ej. `Qwen2.5-1.5B-Instruct`) y `sentence-transformers`. Sin clave, pero requieren descarga.

Reglas de compatibilidad

R1. Mismo modelo para indexar y consultar. Los vectores de documentos y consultas deben salir del *mismo* modelo de embeddings y con los *mismos* parámetros (`input_type` o prefijos, `dimensions`, normalización).

R2. Igual dimensión es necesaria, pero no suficiente. Dos modelos distintos de 384 dimensiones producen espacios vectoriales distintos: el coseno entre ellos no tiene significado.

R3. El LLM de generación es independiente del embedding. Puede cambiar de proveedor de chat sin reindexar; no puede cambiar de modelo de embeddings sin reindexar todo el corpus.

R4. Parámetros específicos de proveedor. `extra_body` de NVIDIA o `logprobs` pueden dar error 400 en otro proveedor: envuélvalos en una función por proveedor.

Archivos de la carpeta `datos_examen/` (en el mismo Drive que este enunciado):

- Sección A: `yolo_results_scratch.csv`, `yolo_results_pretrained.csv`, `yolo_results_freeze.csv`, `yolo_test_preds.json`, `yolo_test_gt.json`.
- Sección C: `embedding_info.json`, `corpus_reglamento.json`, `preguntas_eval.json`, `corpus_embeddings.npy`, `question_embeddings.npy`, `faithfulness_cases.json` (respaldo si no tiene acceso a ningún proveedor).

Sección	A	B	C	D	**Total**
Puntos	33	30	32	5	100
Obtenidos					

Sección A — CNN y detección de objetos con YOLO (33 pts)

En clase entrenó un detector YOLO con transfer learning (notebook `Lecture_08_Yolo_training`). Aquí se entregan los resultados de **tres entrenamientos** con los mismos datos, semilla, épocas y tamaño de imagen: `scratch` (sin pesos preentrenados), `pretrained` (fine-tuning completo) y `freeze` (preentrenado con `freeze=10`).

A.1 — Dimensiones y parámetros (5 pts)

1. (1 pt) Implemente `conv_out(W,K,P,S)` con la fórmula de clase.
2. (2 pts) Valídela con `assert` contra `torch.nn.Conv2d` en **3 configuraciones** (incluya una con stride 2 y una con padding).
3. (2 pts) Construya en PyTorch la red de la **Pregunta 1 de la Parte I**, imprima la forma de salida de cada capa y el número total de parámetros, y compárelos con su cálculo a mano.

A.2 — Comparación de las tres estrategias (12 pts)

Cargue los tres `.csv` con `pandas` (los nombres de columna de Ultralytics pueden traer espacios: use `df.columns = df.columns.str.strip()`).

1. (4 pts) Grafique, para los tres modelos en una misma figura, la **pérdida de caja de entrenamiento** y el **mAP50** por época.
2. (3 pts) Tabla con los valores **finales** de mAP50, mAP50-95, precision, recall y tiempo total de entrenamiento.
3. (5 pts) **Análisis** (máx. 150 palabras): explique el orden de desempeño con evidencia de *sus* gráficas y tabla. ¿Qué estrategia usaría con un dataset 10 veces mayor y de un dominio muy distinto al de preentrenamiento, y por qué? Relacione con la Clase 8.

A.3 — IoU, NMS y evaluación manual (16 pts)

`yolo_test_preds.json` contiene, por imagen (`image_id` de 0 a 29), una lista de predicciones (diccionarios con claves `box` = `[x1,y1,x2,y2]`, `cls` y `conf`) **sin NMS y con confianza baja**; `yolo_test_gt.json` contiene las cajas reales (claves `box` y `cls`). Las cajas están en píxeles, formato `xyxy`. Use las **10 imágenes** con `image_id % 3 == d % 3`, donde d es su último dígito de documento.

1. (5 pts) Implemente `iou(a,b)` y `nms(boxes,scores,iou_thr,score_thr)` (*greedy*, como en clase). Pruébelas con `assert`: $(2,2,6,6)$ vs. $(4,4,8,8)$ debe dar IoU ≈ 0.143, y diseñe un caso de NMS con 4 cajas que elimine al menos un duplicado.
2. (3 pts) Aplique su `nms` (`iou_thr=0.5`) **por clase** a las predicciones de sus 10 imágenes. Reporte cuántas cajas había antes y después.
3. (6 pts) Con las cajas resultantes, empareje con el ground truth (**IoU** > 0.5 **y misma clase**; cada caja real se empareja una sola vez) y calcule **TP, FP, FN, precision y recall** para `conf=0.25` y `conf=0.5`. Presente una tabla.
4. (2 pts) Explique por qué al subir el umbral de confianza sube la precision y baja el recall, usando *sus* números.

Sección B — Transformers y NLP (30 pts)

B.1 — Atención causal desde cero (12 pts)

1. (6 pts) Implemente en PyTorch `causal_self_attention(X,Wq,Wk,Wv)` de **una cabeza**, sin usar `nn.MultiheadAttention` ni funciones de atención ya hechas. Genere `X` (5 tokens, dimensión 8) y las

matrices W (8×4) con su `SEED`.

2. (3 pts) Pruebas automáticas con `assert`: (i) las filas de la matriz de atención suman 1; (ii) el triángulo superior es 0; (iii) la salida coincide (tolerancia 10^{-5}) con `F.scaled_dot_product_attention(..., is_causal=True)`.

3. (3 pts) Reproduzca con su función el ejemplo de la **Pregunta 4 de la Parte I** (pase directamente Q, K, V) y compare con el resultado que obtuvo a mano.

B.2 — Generación autorregresiva con un modelo real (10 pts)

Con `gpt2` (o `distilgpt2`) de Hugging Face, **ejecutado localmente** (esta sección necesita los *logits* completos; Groq rechaza `logprobs` y los endpoints de chat no devuelven la distribución completa):

1. (3 pts) Para el prompt *"The dog chased the"* muestre los 10 tokens más probables con su probabilidad.

2. (4 pts) Aplique temperaturas $T \in \{0.2, 1.0, 2.0\}$ a los logits y reporte para cada una la probabilidad del token top-1 y la **entropía** de la distribución.

3. (3 pts) Explique qué observa y relacione con el cálculo de la Pregunta 5(b) de la Parte I.

B.3 — Tokenización (8 pts)

Con `tiktoken` (`cl100k_base`), compare tokens, caracteres y tokens por carácter para estos tres textos:

EN: *Large language models predict the next token given the previous ones.*

ES: *Los modelos de lenguaje grandes predicen el siguiente token a partir de los anteriores.*

Código: `def f(x): return [i**2 for i in range(x) if i % 2 == 0]`

1. (4 pts) Tabla con los tres conteos y la razón tokens/carácter.

2. (1 pt) Muestre los IDs y el *decode* de un emoji de su elección.

3. (3 pts) Explique en 4–5 líneas por qué el español consume más tokens por carácter y qué implica para costo y ventana de contexto.

Sección C — RAG: compatibilidad, recuperación, búsqueda híbrida e índices (32 pts)

Se entrega un corpus (`corpus_reglamento.json`, mismo formato que `company_documents.json` del laboratorio de RAG) con sus **embeddings ya calculados y normalizados** (`corpus_embeddings.npy`, mismo orden que el corpus), generados con el **modelo oficial de embeddings** descrito en `embedding_info.json` (nombre, dimensión, `input_type` o prefijos y normalización). También se entregan 13 preguntas (`preguntas_eval.json`): 12 de evaluación (`question`, `gold_doc_id`, `tipo` ∈ {`sigla`, `parafrasis`}) y una `demo_fuera_corpus` cuya respuesta **no** está en el corpus; sus embeddings están en `question_embeddings.npy` (mismo orden). **C.1 a C.4 no requieren calcular embeddings nuevos**; C.0 le pide demostrar que sabe hacerlo de forma compatible.

C.0 — Configuración y verificación de compatibilidad (6 pts)

1. (2 pts) Escriba `make_chat_client(provider)` para NVIDIA, Groq o Hugging Face (SDK `openai`, `base_url` de la tabla, clave desde Secrets; sin parámetros específicos de un proveedor en la llamada común). Declare en una celda proveedor y modelo de chat, y haga un *ping* de una frase mostrando la respuesta.

2. (2 pts) Cargue `embedding_info.json`. Verifique con `assert` que la dimensión coincide con `corpus_embeddings.shape[1]` y que las filas están normalizadas. Recalcule el embedding de **un**

documento del corpus con el modelo oficial y los mismos parámetros, y demuestre con el coseno (≥ 0.99) que es compatible con el vector almacenado.

3. (2 pts) Contraejemplo: embeba ese mismo texto con **otro** modelo (preferiblemente de la misma dimensión) o con el oficial pero distinto `input_type`/prefijo, y reporte el coseno contra el vector almacenado. Explique en 2–3 líneas qué le pasaría a la búsqueda si indexara con un modelo y consultara con otro (R1 y R2).

C.1 — Búsqueda vectorial exacta (6 pts)

1. (3 pts) Implemente `retrieve(q_vec,k)` con similitud del coseno mediante multiplicación matricial (sin FAISS).

2. (3 pts) Calcule **Recall@3** y **MRR** sobre las 12 preguntas de evaluación.

C.2 — BM25 y búsqueda híbrida (10 pts)

1. (2 pts) Implemente **BM25** con `rank_bm25` sobre el texto del corpus.

2. (2 pts) Implemente la fusión híbrida con **Reciprocal Rank Fusion**: $\text{score}(d) = \sum_r \dfrac{1}{60 + \text{rank}_r(d)}$.

3. (3 pts) Tabla de Recall@3 de los **tres métodos** (vectorial, BM25, híbrida), **separada por `tipo`** (sigla / parafrasis).

4. (3 pts) Interprete: ¿qué método ganó en cada tipo y por qué? Relacione con la Clase 10. Si algún resultado contradice lo esperado, dígalo y proponga una explicación.

C.3 — Índices aproximados HNSW (5 pts)

Con `faiss-cpu`, genere (con su `SEED`) **20 000 vectores aleatorios normalizados** con la **misma dimensión D de su corpus** y 100 consultas.

1. (3 pts) Compare `IndexFlatIP` (exacto) con `IndexHNSWFlat` (`M=16`) para `efSearch` $\in \{8, 64\}$: **recall@10** (tomando el exacto como verdad), latencia media por consulta y tiempo de construcción.

2. (2 pts) Explique el *trade-off* recall–latencia con *sus* números. Los vectores aleatorios son un caso difícil para HNSW: ¿lo nota?

C.4 — RAG de punta a punta con el LLM de su elección (5 pts)

Con el cliente de C.0 y el *prompt template* del laboratorio, responda usando como contexto los **3 chunks** recuperados con su `retrieve` (C.1) para tres preguntas: `demo_fuera_corpus` y dos del set (una `sigla` y una `parafrasis`). El prompt debe exigir responder **solo** con el contexto y abstenerse si la respuesta no está.

1. (3 pts) Para cada respuesta, en una tabla Markdown liste las afirmaciones, marque cuáles están respaldadas por el contexto y calcule la **Faithfulness**.

2. (2 pts) Analice el caso fuera del corpus: ¿el modelo se abstuvo o alucinó? ¿Qué parte del prompt influyó?

Si no tiene acceso a ningún proveedor, use `faithfulness_cases.json` (3 casos ya generados); el ítem vale como máximo 3 de 5 puntos.

Sección D — Declaración de uso de IA y reproducibilidad (5 pts)

Al final del notebook, una celda Markdown con:

1. (1 pt) Herramienta(s) de IA usadas y **2 prompts clave**, más el **proveedor, modelo de chat y**

modelo de embeddings efectivamente usados.

2. (2 pts) **Un caso concreto** en el que la IA produjo algo incorrecto o inadecuado y cómo lo detectó y corrigió.
3. (2 pts) Reproducibilidad: semilla, versiones de librerías clave y que el notebook corre de principio a fin.

Criterios generales (Parte II)

- **Correctitud y ejecución:** el código corre y los resultados son coherentes.
- **Calidad de la interpretación:** los análisis genéricos o que no se apoyen en *sus* resultados reciben como máximo el 40 % del ítem.
- El docente podrá solicitar una **verificación oral breve (3–5 min)** sobre cualquier celda. La incapacidad de explicar su propio código reduce la nota del ítem correspondiente.

— Fin de la evaluación —

— Fin de la Parte II —